



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

Aplicaciones para Comunicaciones en Red

TEMA:

Práctica: FTP simple

ALUMNO:

Erreguin Franco Yair Alejandro

Boleta: 2022630121

PROFESOR:

Moreno Cervantes Axel Ernesto

25 de junio de 2026

Índice

1. Introducción	2
1.1. Separación de Canales de Control y Datos	2
1.2. Sockets TCP en Java	2
1.3. Uso de Búferes y Empaquetado de Datos	2
1.4. Envío de Comandos, Metadatos y Formato de Comunicación	3
1.4.1. Manejo de Metadatos	3
1.4.2. Estructura del Handshake y Comandos	3
1.4.3. Conversión de Datos e	3
2. Desarrollo Técnico	4
2.1. Estructura del Servidor	4
2.2. Implementación de Transferencia con Buffer de 4 KB	6
3. Ejecución y Resultados	7
3.1. Prueba de Handshake Inicial	7
3.2. Ejecución de Comandos y Movimiento de Archivos	7
4. Conclusiones	8
5. Referencias	10

1. Introducción

La transferencia de archivos es una de las tareas fundamentales cuando trabajamos con redes de computadoras. A diferencia de otros protocolos de la capa de aplicación donde todo viaja por la misma conexión, los servicios FTP separan la comunicación en canales independientes[4]. Esta separación permite enviar comandos y recibir respuestas de estado sin que interfieran con la transferencia masiva de datos binarios.

El objetivo de esta práctica es desarrollar en Java una aplicación de red compuesta por un cliente y un servidor que permita gestionar archivos de forma remota. El programa utiliza un canal fijo para los comandos de control en el puerto 7777 y un canal secundario en el puerto 7778 que se abre de forma temporal para la transmisión física de los archivos.

Para lograr un intercambio eficiente, el sistema fragmenta los archivos en paquetes de tamaño controlado por medio de un buffer. Esto permite leer y escribir bloques directamente en el disco duro sin saturar la memoria RAM del equipo.

1.1. Separación de Canales de Control y Datos

El sistema se organiza utilizando dos sockets independientes para evitar conflictos entre las instrucciones y los archivos transmitidos[4]:

- **Canal de Control (Puerto 7777):** Se encarga de enviar cadenas de texto plano con los comandos de la terminal (LS, CD, TOUCH, MKDIR, MV, RM y RMDIR). Este socket se mantiene abierto durante toda la sesión.
- **Canal de Datos (Puerto 7778):** Se inicializa únicamente cuando se solicita una transferencia pesada mediante los comandos **DOWNLOAD** o **UPLOAD**. En cuanto se envían todos los bytes, este socket se cierra inmediatamente para liberar recursos de red.

1.2. Sockets TCP en Java

Para asegurar que los archivos se transmitan de forma completa, ordenada y sin pérdidas, la comunicación se apoya en el protocolo TCP utilizando las clases `ServerSocket` y `Socket` de Java[1]. El servidor se mantiene en un ciclo de escucha permanente a través del método `accept()`. Una vez que el cliente inicia la sesión, el servidor recibe e interpreta las cadenas de texto correspondientes a cada comando, ejecuta la acción en el sistema de archivos y devuelve una confirmación antes de quedar listo para la siguiente instrucción.

1.3. Uso de Búferes y Empaquetado de Datos

Para evitar problemas de falta de memoria al procesar archivos grandes, el código implementa un buffer de lectura y escritura con un tamaño fijo de **4096 bytes (4 KB)**. En lugar de cargar el archivo completo en la memoria del programa, la información se procesa en paquetes pequeños: se leen 4 KB del disco local, se envían por la red a través del flujo del socket y el receptor los recibe al mismo ritmo para escribirlos en el disco de destino. Este tamaño de 4 KB coincide con el tamaño estándar de los bloques físicos de almacenamiento en los sistemas operativos, lo que optimiza las operaciones de entrada y salida (E/S).

1.4. Envío de Comandos, Metadatos y Formato de Comunicación

La comunicación se divide en texto plano para el control y flujos de bytes puros para los archivos, evitando el uso de serialización compleja o formatos pesados como JSON.

1.4.1. Manejo de Metadatos

Tanto las instrucciones del usuario como los metadatos esenciales de los archivos (como el nombre usando `getName()` y el tamaño exacto en bytes obtenido con `length()`) se transmiten como texto plano (*Strings*) por el puerto 7777. Estos datos se concatenan en una sola línea separados por espacios y finalizan con un salto de línea (`\n`). El programa receptor lee esta línea con `'readLine()'` y utiliza el método `'split()'` para separar y procesar de forma independiente cada variable.

1.4.2. Estructura del Handshake y Comandos

Antes de permitir el ingreso de comandos en la interfaz del cliente, los programas realizan un saludo inicial o *Handshake* de texto por el puerto 7777 para verificar que la conexión funciona de forma correcta.

Este mismo formato de texto se usa para coordinar el canal de datos. Por ejemplo, al bajar un archivo, el cliente solicita el recurso enviando la línea de texto:

```
DOWNLOAD notas.txt\n
```

El servidor recibe la petición, mide el archivo en su disco y responde por el mismo canal enviando los metadatos de confirmación:

```
START_DOWNLOAD 15420\n
```

1.4.3. Conversión de Datos e

La traducción de texto a bytes de red ocurre en los límites del programa utilizando las clases de flujo de Java:

$$\text{Texto Java} \xrightarrow{\text{PrintWriter}} \text{Bytes de Red} \xrightarrow{\text{InputStreamReader}} \text{Texto Java} \quad (1)$$

El objeto `PrintWriter` se encarga de convertir los caracteres de control en bytes para inyectarlos en el socket, mientras que `InputStreamReader` los recibe del otro lado para reconstruir el texto legible. Por el contrario, los archivos binarios que viajan por el puerto 7778 se transmiten en crudo; los bytes se copian idénticos desde el disco hacia el canal de red sin sufrir modificaciones ni traducciones, disminuyendo el uso del procesador.

2. Desarrollo Técnico

El sistema se compone de dos programas independientes en Java: el cliente, que proporciona la consola interactiva de comandos al usuario, y el servidor, que gestiona el directorio raíz de almacenamiento.

2.1. Estructura del Servidor

El servidor procesa el bucle de comandos e interactúa con el sistema de archivos local mediante métodos estáticos encargados de las tareas de creación, borrado y renombrado de inodos.

```
1 package Servidor;
2
3 import java.io.*;
4 import java.net.InetSocketAddress;
5 import java.net.ServerSocket;
6 import java.net.Socket;
7 import java.util.zip.ZipEntry;
8 import java.util.zip.ZipOutputStream;
9
10 public class Servidor {
11
12     private static final int PUERTO_CONTROL = 7777;
13     private static final int PUERTO_DATOS = 7778;
14     private static final String DIR_BASE = "./Servidor/Archivos";
15     private static String dirActual = DIR_BASE;
16
17     public static void main(String[] args) {
18         new File(DIR_BASE).mkdirs();
19
20         try (ServerSocket serverControl = new ServerSocket()) {
21             serverControl.setReuseAddress(true);
22             serverControl.bind(new InetSocketAddress(PUERTO_CONTROL));
23
24             while (true) {
25                 dirActual = DIR_BASE;
26                 refrescarConsola(dirActual, "ESPERANDO", "Cliente...", "
27                     0.0.0.0");
28
29                 try (Socket socketCliente = serverControl.accept()) {
30                     String clientIp = socketCliente.getInetAddress().
31                         getHostAddress();
32
33                     try (
34                         BufferedReader in = new BufferedReader(new
35                             InputStreamReader(socketCliente.getInputStream()
36                                 ());
37                         PrintWriter out = new PrintWriter(socketCliente.
38                             getOutputStream(), true)
39                     ) {
40                         out.println("CONEXION_OK");
41                     }
42                 }
43             }
44         }
45     }
46 }
```

```

36         refrescarConsola(dirActual, "HANDSHAKE", "
37             CONEXION_OK", clientIp);
38
39         String linea;
40         while ((linea = in.readLine()) != null) {
41             String[] partes = linea.split(" ", 2);
42             String comando = partes[0].toUpperCase();
43             String argumento = partes.length > 1 ? partes
44                 [1] : "";
45
46             if (comando.equals("EXIT")) {
47                 out.println("CHAO");
48                 break;
49             }
50
51             switch (comando) {
52                 case "LS": listarArchivos(argumento, out);
53                     break;
54                 case "CD": cambiarDirectorio(argumento,
55                     out); break;
56                 case "TOUCH": crearArchivoRemoto(argumento
57                     , out); break;
58                 case "MKDIR": crearCarpetaRemota(argumento
59                     , out); break;
60                 case "MV": renombrarElementoRemoto(
61                     argumento, out); break;
62                 case "RM": eliminarArchivoRemoto(argumento
63                     , out); break;
64                 case "RMDIR": eliminarCarpetaRemota(
65                     argumento, out); break;
66                 case "DOWNLOAD": enviarElementoAlCliente(
67                     argumento, out); break;
68                 case "UPLOAD": recibirElementoDelCliente(
69                     argumento, out); break;
70                 default: out.println("ERROR: Comando
71                     invalido."); break;
72             }
73             refrescarConsola(dirActual, comando, argumento
74                 , clientIp);
75         }
76     } catch (IOException e) {
77         System.out.println("Conexion con el cliente
78             interrumpida.");
79     }
80     refrescarConsola(dirActual, "DESCONECTADO", "Cierre de
81         Socket", clientIp);
82 }
83
84 }
85
86 } catch (IOException e) {
87     System.err.println("Error critico: " + e.getMessage());
88 }
89
90 }
91
92 // ... Metodos operativos estaticos (listarArchivos, cambiarDirectorio
93     , enviarElementoAlCliente, etc.)

```

Listing 1: Código de la estructura del Servidor

2.2. Implementación de Transferencia con Buffer de 4 KB

Cuando se procesa la recepción de un archivo, los bytes entrantes se leen del flujo de red de manera segmentada utilizando el buffer físico de 4096 bytes antes de grabarse de forma directa en el almacenamiento:

```

1      try (ServerSocket serverDatos = new ServerSocket()) {
2          serverDatos.setReuseAddress(true);
3          serverDatos.bind(new InetSocketAddress(PUERTO_DATOS));
4          try (
5              Socket socketDatos = serverDatos.accept();
6              InputStream is = socketDatos.getInputStream();
7              FileOutputStream fos = new FileOutputStream(destino)
8          ) {
9              byte[] buffer = new byte[4096]; // Buffer de empaquetado de 4
10                 KB
11                 int bytesLeidos;
12                 long totalRecibido = 0;
13                 while (totalRecibido < tamano && (bytesLeidos = is.read(buffer
14                     , 0, (int) Math.min(buffer.length, tamano - totalRecibido))
15                     ) != -1) {
16                     fos.write(buffer, 0, bytesLeidos);
17                     totalRecibido += bytesLeidos;
18                 }
19                 out.println("OK: Elemento subido.");
20             }
21         } catch (IOException e) {
22             out.println("ERROR: Fail");
23         }
24     }

```

Listing 2: Fragmento de código para la recepción de paquetes de datos

3. Ejecución y Resultados

3.1. Prueba de Handshake Inicial

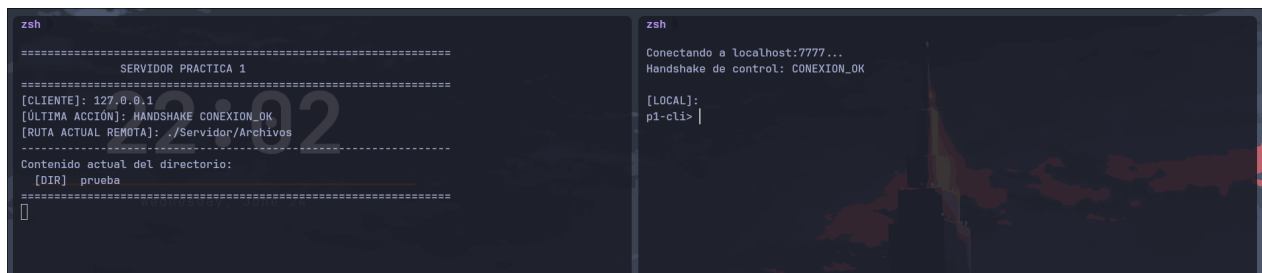
Al poner en marcha el servidor, la terminal se limpia utilizando secuencias de escape ANSI quedando en estado de espera. Cuando iniciamos el programa `Cliente.java`, el canal de control establece la comunicación y despliega en pantalla la confirmación del saludo de sincronía:

```
Conectando de forma activa a localhost:7777...
```

```
Handshake de control: CONEXION_OK
```

```
[LOCAL: ./Cliente/Archivos]
```

```
p1-cli>
```

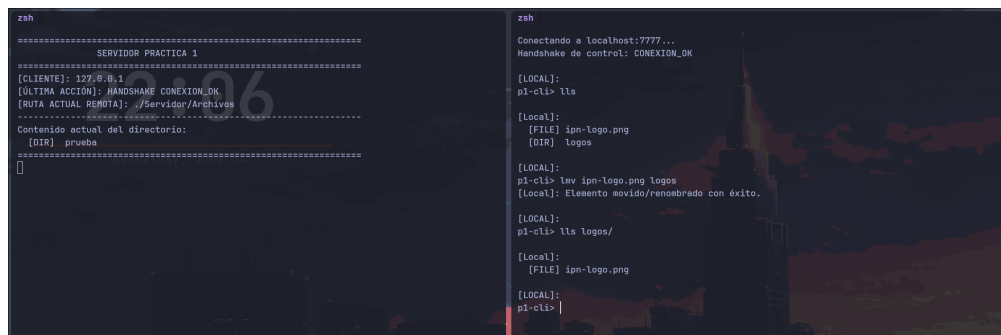


The image shows two terminal windows side-by-side. The left window, titled 'zsh', displays the output of a server program 'SERVIDOR PRACTICA 1'. It shows the client IP as 127.0.0.1, the last action as 'HANDSHAKE CONEXION_OK', and the current remote path as './Servidor/Archivos'. It also shows the contents of the current directory as '[DIR] prueba'. The right window, also titled 'zsh', shows the output of a client program. It displays 'Conectando a localhost:7777...', 'Handshake de control: CONEXION_OK', and the local path '[LOCAL: ./Cliente/Archivos]'. The prompt 'p1-cli>' is visible at the bottom.

Figura 1: Línea de comandos del cliente mostrando el handshake exitoso con el servidor.

3.2. Ejecución de Comandos y Movimiento de Archivos

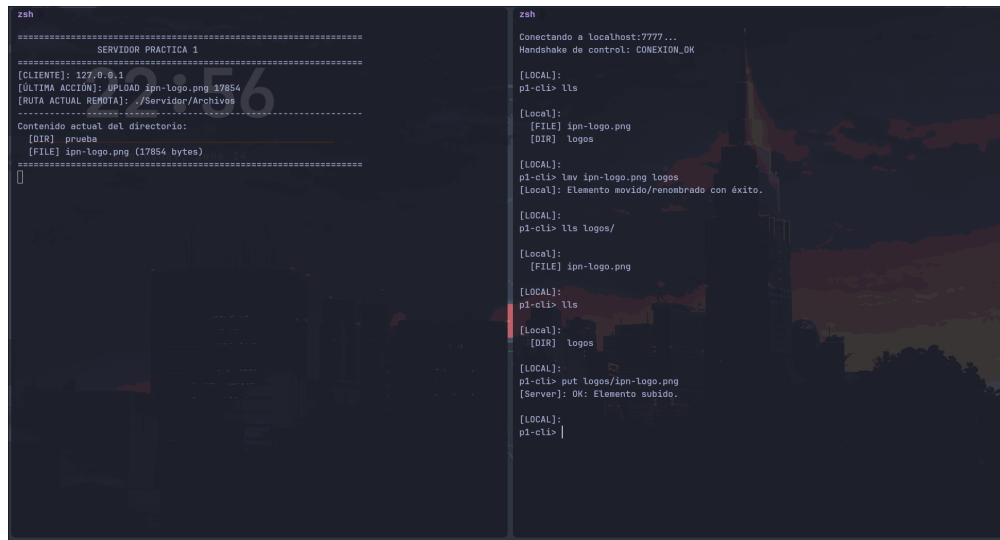
El programa permite listar carpetas locales utilizando `lls` pasándole un argumento de ruta relativa. Si ejecutamos el comando de movimiento local `lmv` indicando un archivo de origen y una carpeta de destino existente, el código calcula la ruta de forma automática para reubicar el archivo dentro de ese directorio.



The image shows two terminal windows side-by-side. The left window, titled 'zsh', displays the output of a server program 'SERVIDOR PRACTICA 1'. It shows the client IP as 127.0.0.1, the last action as 'HANDSHAKE CONEXION_OK', and the current remote path as './Servidor/Archivos'. It also shows the contents of the current directory as '[DIR] prueba'. The right window, also titled 'zsh', shows the output of a client program. It displays 'Conectando a localhost:7777...', 'Handshake de control: CONEXION_OK', and the local path '[LOCAL: ./Cliente/Archivos]'. The prompt 'p1-cli>' is visible at the bottom.

Figura 2: Resultados en la consola al listar el contenido de subcarpetas y mover archivos.

A continuación se muestra el uso del comando **put** el cual sube un archivo al servidor desde el directorio local:



```
zsh
=====
SERVIDOR PRACTICA 1
=====
[CLIENTE]: 127.0.0.1
[ÚLTIMA ACCIÓN]: UPLOAD ipn-logo.png 17854
[RUTA ACTUAL REMOTA]: ./Servidor/Archivos
-----
Contenido actual del directorio:
[DIR] prueba
[FILE] ipn-logo.png (17854 bytes)
=====
[]

zsh
Conectando a localhost:7777...
Handshake de control: CONEXION_OK

[LOCAL]:
pi-cli> ls
[LOCAL]:
[FILE] ipn-logo.png
[DIR] logos
[LOCAL]:
pi-cli> mv ipn-logo.png logos
[LOCAL]: Elemento movido/renombrado con éxito.
[LOCAL]:
pi-cli> ls logos/
[LOCAL]:
[FILE] ipn-logo.png
[LOCAL]:
pi-cli> ls
[LOCAL]:
[DIR] logos
[LOCAL]:
pi-cli> put logos/ipn-logo.png
[Server]: OK: Elemento subido.
[LOCAL]:
pi-cli> |
```

Figura 3: Resultados en la consola al subir un archivo al servidor.

De igual manera se muestra el funcionamiento del comando **get** el cual recupera un archivo del servidor al cliente:



```
zsh
=====
SERVIDOR PRACTICA 1
=====
[CLIENTE]: 127.0.0.1
[ÚLTIMA ACCIÓN]: DOWNLOAD prueba
[RUTA ACTUAL REMOTA]: ./Servidor/Archivos
-----
Contenido actual del directorio:
[DIR] prueba
[FILE] ipn-logo.png (17854 bytes)
=====
[]

java Cliente/Cliente
+ pi git:(main) * java Cliente/Cliente
Conectando a localhost:7777...
Handshake de control: CONEXION_OK

[LOCAL]:
pi-cli> get prueba
[Datos]: Descarga finalizada (17295 bytes).
[LOCAL]:
pi-cli> |
```

Figura 4: Resultados en la consola al bajar un archivo al servidor.

Cuando el usuario escribe **exit**, el bucle del cliente termina su ejecución de forma limpia, y la pantalla del servidor se actualiza al estado de desconexión, reiniciando su ruta base para quedar listo ante una nueva sesión.

4. Conclusiones

El desarrollo de esta práctica me ayudó a comprender de forma clara y práctica el diseño de protocolos en la capa de aplicación. Separar las tareas del programa utilizando dos canales independientes (puerto 7777 para control y puerto 7778 para datos) es una solución excelente, ya que evita que las instrucciones de texto interfieran con el flujo binario continuo de los archivos al subirlos o bajarlos.

Por otro lado, implementar el búfer fijo de 4096 bytes (4 KB) me enseñó a optimizar el manejo de recursos y la persistencia en disco. Transmitir la información en bloques pequeños del tamaño exacto de las páginas del sistema operativo evita saturar la memoria RAM del

sistema y acelera la lectura y escritura. Finalmente, programar la lógica del sistema para comandos como `mv`, `rmv` o `ls` remotos de modo que reconozcan argumentos de carpetas y organicen los inodos de forma automática me dio una visión real de cómo operan los sistemas de archivos en red distribuidos.

En esta práctica además se observa como los datos viajan a través de la red sin protección, esto no ocurre así en la vida real, generalmente estos datos se cifran con un algoritmo simétrico usando una clave acordada entre el cliente y el servidor.

5. Referencias

Referencias

- [1] Oracle, “Class ServerSocket,” Java Standard Edition, 2025. [En línea]. Disponible: <https://docs.oracle.com/en/java/javase/>. [Accedido: 24-jun-2026].
- [2] Jsoup, “jsoup: Java HTML Parser.” [En línea]. Disponible: <https://jsoup.org/>. [Accedido: 24-jun-2026].
- [3] GeeksforGeeks, “What is a Web Crawler and where is it used?,” 2024. [En línea]. Disponible: <https://www.geeksforgeeks.org/>. [Accedido: 24-jun-2026].
- [4] Mozilla Developer Network, “An overview of HTTP y FTP.” [En línea]. Disponible: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>. [Accedido: 24-jun-2026].